گزارش پروژه: سرور تراکنش بانکی توزیع‌پذیر (Mini Banking Server)

این مستند، گزارش کامل طراحی، پیاده‌سازی و اعتبارسنجی سامانه تراکنش بانکی توزیع‌شده با تضمین کامل خواص ACID، مدیریت کلاینت‌های همزمان، تحمل‌پذیری در برابر خرابی با مکانیزم انتقال خودکار ترافیک (Failover) و جلوگیری از دوپارگی مغز در زمان قطع شبکه بر اساس الگوریتم حدنصاب اکثریت (Split-Brain Prevention) است.

۱. روش ارتباطی و پروتکل تبادل داده (Communication Method)

برای برقراری ارتباط بین ماشین‌های کلاینت و سرور، از پروتکل **HTTP/1.1 با ساختار داده‌ای استاندارد JSON** در بستر فریم‌ورک سبک و پرسرعت **FastAPI** و سرور ASGI به نام **Uvicorn** استفاده شده است. ماشین‌های سرور و کلاینت روی ماشین‌های مجازی (Virtual Machines) مجزا در شبکه Host-Only اجرا شده‌اند و کلاینت مستقیماً به آدرس IP واقعی سرورها متصل می‌شود (نه localhost).

ساختار یکنواخت درخواست‌ها و پاسخ‌ها به صورت زیر است:

```
{ "action": "withdraw", "account": "A1001", "amount": 200 }
```

```
{ "status": "success", "account": "A1001", "balance": 800, "request_id": "a1b2c3d4" }
```

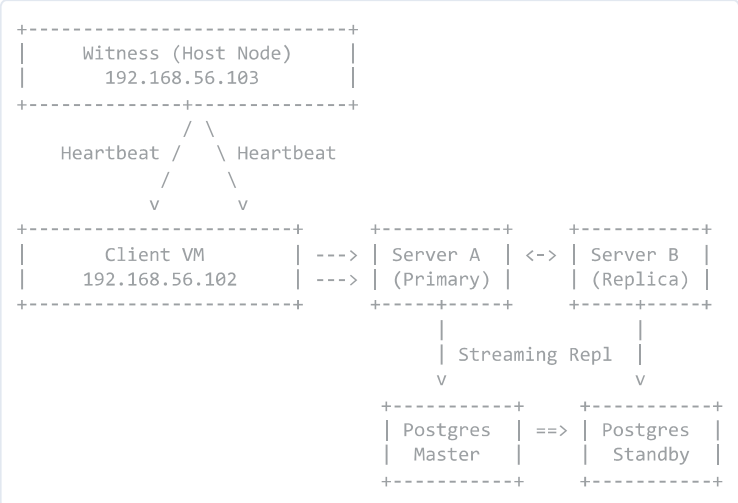
پاسخ‌های خطا نیز از یک مدل استاندارد و یکپارچه همراه با کد وضعیت HTTP معنادار تبعیت می‌کنند:

```
{ "status": "error", "message": "insufficient funds", "account": "A1001" }
```

۲. معماری کلاستر توزیع‌شده و نگاشت شبکه

کلاستر از سه ماشین مجازی لینوکسی (Ubuntu) مجزا و یک گره ناظر (Witness) روی سیستم‌عامل میزبان تشکیل شده است:

- **Witness (گره داور/ناظر):** اجرا روی هاست فیزیکی به آدرس `192.168.56.103:9000` جهت رأی‌گیری حدنصاب
- **Server A (سرور اصلی / Primary):** اجرا روی VM سرور اول به آدرس `192.168.56.101:8000`
- **Server B (سرور پشتیبان / Hot Standby):** اجرا روی VM سرور دوم به آدرس `192.168.56.104:8000`
- **Client VM (کلاینت):** اجرا روی VM کلاینت به آدرس `192.168.56.102`



۳. مدیریت کلاینت‌های همزمان (Concurrent Clients)

سرور به ازای هر درخواست دریافتی یک Handler در تردهای مجزا اجرا می‌کند. جهت ممانعت از اشباع منابع و اضافه بار سیستم، سقف کلاینت‌های پردازش‌شونده به صورت همزمان از طریق متغیر محیطی `MAX_CLIENTS` (به عنوان پیش‌فرض

۲۰) کنترل می‌شود. این ویژگی با کمک ساختار همگام‌ساز `threading.Semaphore` بدون مسدودسازی (Non-blocking: `acquire(blocking=False)`) پیاده‌سازی شده است:

- در صورتی که ظرفیت خالی موجود باشد، درخواست وارد چرخه پردازش تراکنش می‌شود.
- در صورت اشباع بودن سرور، پردازش بلافاصله با کد `HTTP 503 (Server Busy)` رد شده و جزئیات آن لاگ می‌شود.

۴. مدیریت وضعیت مشترک (Shared State Management)

اطلاعات حساب‌ها و موجودی آن‌ها به عنوان **وضعیت مشترک (Shared State)**، به هیچ وجه در متغیرهای فرار رم سرور نگهداری نمی‌شوند؛ بلکه مستقیماً در پایگاه داده رابطه‌ای **PostgreSQL** ذخیره می‌گردند. این معماری تضمین می‌کند که:

- هر دو سرور A و B یا هر چند نود ناهمگام دیگر همواره با دسترسی به پایگاه داده، دیتای حقیقی را مشاهده کنند.
- با خاموش شدن ناگهانی یا ری‌استارت کامل پردازش سرور، موجودی‌ها دست‌نخورده روی دیسک باقی بمانند.
- همگام‌سازی بین سرورها به پروتکل‌های دست‌نویس متکی نبوده و با مکانیسم بهینه **Streaming Replication** خود پایگاه داده بر بستر ارسال مداوم لاگ‌های WAL انجام شود.

```
CREATE TABLE IF NOT EXISTS accounts (
    account_id VARCHAR(32) PRIMARY KEY,
    balance    NUMERIC(14, 2) NOT NULL CHECK (balance >= 0)
);
```

۵. حفظ صحت عملیات همزمان روی یک حساب (Concurrency & Row-Level Locks)

اگر دو تراکنش واریز یا برداشت در یک میلی‌ثانیه برای حساب `A1001` دریافت شوند، چه رخ می‌دهد؟ برای جلوگیری از ناهنجاری‌های معروفی چون Lost Update یا Dirty Read، از مکانیزم قفل‌گذاری سطری (Row-Level Locking) با دستور استاندارد `SELECT ... FOR UPDATE` استفاده شده است:

1. تراکنش اول با دستور فوق، سطر حساب را قفل کرده و موجودی را بازخوانی می‌کند.
2. تراکنش دوم که به طور موازی به حساب مراجعه کرده، پشت قفل سطر در سطح پایگاه داده متوقف می‌شود.
3. تراکنش اول عملیات ریاضی را انجام داده، دستور `UPDATE` را ثبت کرده و تراکنش را `COMMIT` می‌کند؛ قفل آزاد می‌شود.
4. تراکنش دوم بلافاصله ادامه یافته و سطر را با **موجودی جدید و به‌روزشده** می‌خواند.

در نتیجه، خروجی همواره منطبق بر یک ترتیب سریال‌پذیر (Serializability) قطعی است. در استعلام موجودی (اکشن `balance`)، جهت حذف سربار قفل‌گذاری و امکان خواندن بر روی دیتابیس‌های رپلیکای Standby، از `SELECT` ساده بدون قفل استفاده می‌شود.

۶. تضمین خواص چهارگانه تراکنش (ACID Guarantees)

ویژگی	مفهوم و راهکار تحقق در پیاده‌سازی سیستم
Atomicity (تجزیه‌ناپذیری)	تراکنش یا به‌طور کامل اجرا می‌شود یا هیچ اثری از آن باقی نمی‌ماند. تمامی عملیات درون یک مدیریت‌کننده کانتکست (`with conn.transaction():`) اجرا می‌شوند. در صورت بروز هرگونه خطای منطقی (موجودی ناکافی، حساب ناموجود) یا کرش میانه راه، وضعیت به طور خودکار به نقطه آغاز `ROLLBACK` می‌شود.
Consistency (سازگاری)	حساب‌ها هرگز وضعیت نامعتبر نمی‌گیرند. این شرط در دو لایه پیاده شده است: قید سیستمی `CHECK (balance >= 0)` در سطح دیتابیس و اعتبارسنجی دقیق مقدار عددی ورودی با نوع داده دقیق `Decimal` (مبالغ باید مثبت و حداکثر دو رقم اعشار باشند). هیچ پولی خلق یا محو نمی‌شود.
Isolation (جداسازی)	تراکنش‌های همزمان نمی‌توانند دیتای ناتمام یکدیگر را ببینند. با بهره‌گیری از قفل‌گذاری سطری `SELECT ... FOR UPDATE` در سطح ایزولاسیون استاندارد Read Committed، تداخل‌های همزمانی کاملاً ایزوله شده و اجرای موازی با اجرای ترتیبی یک‌به‌یک معادل می‌گردد.

ویژگی	مفهوم و راهکار تحقق در پیاده‌سازی سیستم
Durability (ماندگاری)	تغییرات تأیید شده هرگز از بین نمی‌روند. تراکنش قبل از اینکه پاسخ HTTP 200 به سمت کلاینت بازگردد، روی Write-Ahead Log (WAL) دیسک پایگاه داده `COMMIT` می‌شود. در صورت قطع برق یا کشته شدن پروسه سرور بلافاصله پس از پاسخ، داده‌ها سالم بازخوانی می‌شوند.

۷. تحمل‌پذیری در برابر خرابی و Failover خودکار

در سناریوی پیشرفته، سامانه به گونه‌ای طراحی شده که با متوقف شدن یا سوختن ناگهانی سرور اصلی (Server A)، کلاینت‌ها بدون کوچک‌ترین وقفه به سرویس‌دهی ادامه دهند:

- **پایگاه داده همگام (Streaming Replication):** دیتابیس سرور B با ابزار `pg_basebackup` به عنوان رپلیکای بلادرنگ سرور A راه‌اندازی شده و تمام لاگ‌های تراکنش را لحظه‌ای دریافت می‌کند.
- **انتقال شفاف سمت کلاینت (Client-Side Failover):** در کلاینت لیستی از سرورها (`SERVER_URLS=[A, B]`) تعریف شده است. کلاینت همیشه ابتدا سرور A را فراخوانی می‌کند؛ در صورتی که اتصال با خطای سوکت، Timeout، یا کدهای خطای سرور ۵۰۲/۵۰۳/۵۰۴ مواجه شود، بلافاصله در همان متد `call()` درخواست را به سرور B هدایت می‌کند.
- **ارتقای خودکار پایگاه داده (Automatic Promotion):** سرور B حین دریافت تراکنش‌های نوشتن به عنوان سرور لیدر، وضعیت دیتابیس را بررسی می‌کند؛ اگر دیتابیس در حالت Standby باشد، با دستور سیستمی `pg_promote()` آن را بدون نیاز به مداخله دست به Master ارتقا می‌دهد تا تراکنش‌های واریز/برداشت با موفقیت درج شوند.
- **بازگشت سالم نود خراب‌شده (Rejoin):** برنامه با مقایسهٔ نقش و `timeline` پایگاه‌داده‌ها نود عقب‌تر را تشخیص می‌دهد، درخواست‌های آن را با ۵۰۳ رد می‌کند و با `rejoin_as_replica.sh` به رپلیکای سرور B تبدیل می‌کند؛ ارتقای خودکار نیز فقط زمانی رخ می‌دهد که همتای primary زنده نباشد.

۸. مقابله با قطع شبکه و جلوگیری از split brain

مسئله Split-Brain: اگر کابل یا شبکه ارتباطی میان Server A و Server B قطع شود، اما هر دو ماشین روشن بمانند، ممکن است هر دو سرور تصور کنند طرف مقابل مرده است و خود را سرور اصلی پندارند. در این حالت، کلاینت‌ها روی هر دو سرور تراکنش ثبت کرده و موجودی حساب‌ها دچار انشعاب و تناقض اصلاح‌ناپذیر می‌شود.

راهکار مبتنی بر حدنصاب اکثریت (Quorum-Based Consensus): یک کلاستر ۳ عضوی تعریف شده است: `[Server A, Server B, Witness]` . حدنصاب لازم برای تصمیم‌گیری و ثبت تراکنش، اکثریت مطلق است:

```
Quorum = floor(N / 2) + 1 = 2 out of 3 votes
```

قبل از اجرای هر عملیات واریز یا برداشت، تابع `has_quorum()` وضعیت اتصال با دو عضو دیگر را در بازه ۲ ثانیه‌ای می‌سنجد:

- **سمت اکثریت:** سروری که به گره Witness متصل است (۲ رأی از ۳ رأی)، حدنصاب داشته و تراکنش‌های نوشتن را می‌پذیرد.
- **سمت اقلیت:** سروری که ایزوله شده (۱ رأی از ۳ رأی)، سریعاً به حالت **فقط‌خواندنی (Read-Only)** تغییر وضعیت می‌دهد و درخواست‌های نوشتن را با خطای `HTTP 503 (no quorum: read-only mode)` ریجکت می‌کند تا از دوپارگی اطلاعات ممانعت شود. استعلام موجودی (`balance`) همچنان روی این سرور بدون خطا پاسخ داده می‌شود.
- **همگام‌سازی پس از بازگشت شبکه (Healing):** پس از برطرف شدن قطعی شبکه، replication دیتابیس ادامه‌ی لاگ‌های WAL را دریافت کرده و هر دو نود به توازن کامل بازمی‌گردند. بر مبنای قضیه CAP، در زمان پارتیشن، مدل سازگاری بر دسترسی‌پذیری ترجیح داده شده است (CP).
- **تکمیل با Lease انحصاری (اختیاری):** شمارش رأی به‌تنهایی اگر فقط لینک A↔B قطع شود و هر دو سرور Witness را ببینند، می‌تواند به دو Primary هم‌زمان منجر شود. با `ENABLE_LEASE=true` ، گره Witness یک اجاره‌نامه‌ی زمانی انحصاری به فقط یک سرور می‌دهد و سرور تنها تا زمان اعتبار اجاره اجازه‌ی نوشتن دارد؛ در غیر این صورت فوراً به حالت فقط‌خواندنی می‌رود (Fail-Closed).

۹. مدیریت جامع خطاها و لاگ‌گیری استاندارد (Error Handling & Logging)

تمامی خطاهای تعیین‌شده در مشخصات پروژه به شیوه‌ای امن و بدون توقف سرویس پردازش می‌شوند:

نوع خطا	کد HTTP	پیام نمونه / شیوه کنترل
Unknown account (حساب ناموجود)	404	`unknown account: Z9999` (با اعتبارسنجی جستجوی کلید)
Insufficient funds (کسری موجودی)	400	`insufficient funds` (محاسبه و مقایسه در تراکنش)
Negative/Zero/Invalid amount	400	`amount must be a positive number` (پایپ‌لاین Decimal)
Unsupported action (اکشن ناشناخته)	400	`unsupported action: transfer` (فیلتر مقادیر مجاز)
Missing fields (فیلد ناقص)	400	`missing account field` یا `missing amount field`
Malformed JSON (داده خراب/بایت تصادفی)	400	`malformed request` (مدیریت سراسری استثناهای FastAPI)
Server busy / No quorum	503	ترافیک بیش از حد یا پارتیشن شبکه (جهت هدایت کلاینت)

هر درخواست با جزئیات کامل به کنسول و فایل `bank_server.log` فرستاده می‌شود:

```
2026-10-15T14:21:15.278Z  req=acbcbef4  ip=192.168.56.102  action=balance  account=A1001  result=success
balance=50.0  latency=40ms
```

۱۰. راهنمای گام‌به‌گام راه‌اندازی و اجرای کدها (How to Run)

۱. راه‌اندازی گره ناظر (Witness) روی سیستم هاست (ویندوز / PowerShell)

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install fastapi uvicorn
python witness.py
```

۲. راه‌اندازی Server A (سرور اصلی: `192.168.56.101`)

```
# نصب و تنظیم دیتابیس
sudo apt update && sudo apt install -y postgresql postgresql-contrib
sudo -u postgres psql -c "CREATE USER bankuser WITH PASSWORD 'bankpass' SUPERUSER;"
sudo -u postgres psql -c "CREATE DATABASE minibank OWNER bankuser;"
sudo -u postgres psql -c "CREATE USER replicator WITH REPLICATION PASSWORD 'replicapass';"
psql -h 127.0.0.1 -U bankuser -d minibank -f schema.sql

# تنظیم pg_hba.conf و postgresql.conf برای گوش دادن به شبکه و فعال‌سازی replication
sudo systemctl restart postgresql

# اجرای برنامه پایتون در محیط venv
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
# تنظیم فایل .env با آدرس‌های DATABASE_URL، WITNESS_URL و PEER_SERVER_URL
python server.py
```

۳. راه‌اندازی Server B (سرور پشتیبان: `192.168.56.104`)

```
# کلون‌گیری نسخه اولیه از سرور اصلی برای راه‌اندازی Replica
sudo systemctl stop postgresql
sudo rm -rf /var/lib/postgresql/18/main/*
sudo -u postgres pg_basebackup -h 192.168.56.101 -U replicator -D /var/lib/postgresql/18/main -Fp -Xs -P
-R
# تنظیم hot_standby = on در postgresql.conf
sudo systemctl start postgresql

# اجرای سرور B
python3 -m venv venv && source venv/bin/activate
```

```
pip install -r requirements.txt
python server.py
```

۴. اجرای دستورات کلاینت روی ماشین Client VM (192.168.56.102)

```
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# تست استعلام موجودی، واریز و برداشت
python client.py balance A1001
python client.py deposit A1001 100
python client.py withdraw A1001 50

# اجرای تست تمام حالات خطا
python client.py errors

# تست استرس و همزمانی بالا (۲۰ کلاینت همزمان)
python client.py stress A1001 20 10

# دمو انتقال خودکار ترافیک در حین قطع سرور اصلی
python client.py failover-demo A1001
```

۱۱. نتایج آزمون‌ها و اعتبارسنجی ویدیو

- **آزمون عملیات عادی:** کلیه عملیات استعلام، واریز و برداشت با خروجی‌های معتبر عددی انجام شدند.
- **آزمون همزمانی (Stress):** با اجرای ۲۰ برداشت همزمان، خروجی `OK – matches serial expectation` دریافت شد که درستی قفل سطری را اثبات نمود.
- **آزمون Atomicity و Durability:** برداشت ناموفق ۹۹۹۹۹۹ تومانی وضعیت حساب را تغییر نداد، و با کرش دادن سرور و استارت مجدد، کلیه مانده‌ها از روی دیسک دقیق بازخوانی شدند.
- **آزمون Failover خودکار :** حین اجرای حلقه مداوم ۲ requests در ثانیه، سرور A کشته شد؛ کلاینت در درخواست بعدی بدون وقفه به سرور B وصل شد و ادامه داد.
- **آزمون قطع شبکه :** ارتباط با iptables مسدود گردید؛ سرور ایزوله‌شده فاقد حدنصاب تشخیص داده شد و به حالت Read-Only رفت و خطای ۵۰۳ داد، در حالی که سرور واجد حدنصاب به کار ادامه داد و پس از اتصال، دیتابیس‌ها به شکل سازگار همگام شدند.